

Life Sciences Suite

Foundation — Volume 0 & Volume 1

A greenfield implementation and validation guide for the complete Life Sciences Suite running on **Odoo 19 Community Edition**, covering pharmaceutical, medical device, medical plastics, cosmetics and laboratory operations under GMP, ISO and regulatory frameworks.

Document	Foundation (Volume 0 + Volume 1)
Edition	1.0 (Draft for review)
Date	August 2026
Platform	Odoo 19.0 Community Edition · PostgreSQL 16 · Docker
License	LGPLv3 (Odoo Community) · AGPL-3 (OCA & Life Sciences modules)
Classification	Confidential — Enterprise Architecture Document
Truth protocol	VERIFIED / ASSUMPTION / ARCHITECTURAL RECOMMENDATION / CUSTOM DEVELOPMENT

This document is not a substitute for regulatory or legal advice. Software alone does not guarantee regulatory compliance. See the Truth Protocol and Regulatory Disclaimer in Volume 0.

NAVIGATION

Table of Contents

Volume 0 — Front Matter & How to Use This Guide	5
0.1 Document Purpose	5
0.2 Source of Truth	5
0.3 Truth Protocol (Mandatory)	5
0.4 Regulatory Disclaimer	6
0.5 Regulatory Frameworks Referenced	6
0.6 How This Guide Is Structured	6
0.7 Two-Tier Treatment	7
0.8 Edition & Revision Control	7
Volume 1 — Foundation & Architecture	8
1.1 Four-Layer Architecture	8
1.2 Deployment Topology	8
Step 1 — Empty-Database Initialization	9
1. Business Objective	9
2. Business Context	9
3. Why this step is required	9
6. Regulatory Context	9
7. Applicable Industries	9
8. Required Modules	9
10. Prerequisites	10
11. Actors	10
16. Configuration Parameters	10
19. Workflow	10
39. Common Mistakes	11
40. Best Practices	11
47–51. Testing & Validation (Platform IQ)	11
55. Backup Strategy	11
57. References	11
Step 2 — Native Module Baseline Install	12
1. Business Objective	12

3. Why this step is required	12
8. Required Modules & Dependency Order	12
14. Menus & 15. Navigation	12
18. Business Rules	12
39. Common Mistakes	12
47–51. Testing & Validation	13
57. References	13

1.3 OCA Module Selection & Deployment **14**

Verified OCA families present	14
Recommended baseline	14
57. References	14

Step 3 — Company, Sites, Plants, Buildings, Cleanrooms, Warehouses, Locations **15**

1. Business Objective	15
6. Regulatory Context	15
16. Configuration Parameters & 17. Field-by-field	15
18. Business Rules	16
40. Best Practices	16
57. References	16

Step 4 — Departments & Users **17**

1. Business Objective	17
16–17. Fields	17
18. Business Rules	17
39. Common Mistakes	17
57. References	18

Step 5 — Security Architecture: Groups, Roles, Access Rights, Record Rules **19**

1. Business Objective	19
6. Regulatory Context	19
17. Field-by-field — the four security mechanisms	19
32. Security Groups & 33. Record Rules — design pattern	19
40. Best Practices	20
57. References	20

Step 6 — Electronic Signatures **21**

1. Business Objective	21
6. Regulatory Context	21
8. Required Modules	21
12. Roles and Permissions (verified)	21
13. Required Master Data	21
14–15. Menus & Navigation	22
16–17. Configuration & Field-by-field — Signature Meaning	22
18. Business Rules (verified)	22
22–23. Electronic Signatures & Audit Trail	22
24. Notifications	23
25–27. Documents, Records, Reports	23
47–52. Testing & CSV	23
39. Common Mistakes	23
57. References	23

Step 7 — Audit Trail

24

1. Business Objective	24
6. Regulatory Context	24
8. Required Modules	24
12. Roles and Permissions (verified)	24
13. Required Master Data	24
16–17. Configuration & Field-by-field — Audit Rule	25
18. Business Rules (verified)	25
24. Notifications & 26. Generated Records	25
27. Reports	26
47–52. Testing & CSV	26
39. Common Mistakes	26
40. Best Practices	26
55–56. Backup & Disaster Recovery	26
57. References	26

VOLUME 0

Volume 0 — Front Matter & How to Use This Guide

0.1 Document Purpose

This guide is the operational and validation reference for the Life Sciences Suite, an ERP solution built on **Odoo 19 Community Edition** for organisations operating in pharmaceutical manufacturing, medical devices, medical plastics, cosmetics and laboratories. It is written so that a new company can configure the system from an empty database through to commercial production, in the correct chronological order, without skipping mandatory prerequisites.

The guide targets ERP consultants, system integrators, validation teams, QA departments, regulatory affairs, auditors, IT departments and production managers. It is structured to be comparable in quality and depth to validated pharmaceutical ERP implementation documentation produced by major consulting and software houses.

0.2 Source of Truth

The primary references for this guide are, in order of precedence:

1. **The supplied Life Sciences Suite Functional Specification** (v1.0, July 2026) — the authoritative functional reference.
2. **The supplied Life Sciences Suite Architecture document** — the four-layer architecture model (Native Odoo → OCA → Life Sciences Core → Industry).
3. **The actual codebase** under `addons/` in the project root — module manifests, models, security files, record rules and demo data. Where the code differs from a summary, the code is truth.
4. **Odoo 19 Community** documentation and the **OCA** official repositories at <https://github.com/OCA>.
5. **Official regulations only** — cited inline wherever a regulatory claim is made.

VERIFIED AGAINST CODEBASE

The codebase at `D:\Docker\odoo19Claude_ls-docker` contains 442 addon folders. Of these, **17 Life Sciences Core modules** (`ls_*`) and **1 Industry module** (`ls_pharma`) are present with complete manifests, security files, record rules, views, reports, wizards, demo data and cron jobs at version **19.0.1.0.0**. Four industry modules specified in the Functional Specification are **not present** in the codebase: `ls_medical_device`, `ls_medical_plastics`, `ls_cosmetics` and `ls_lab`. These are documented as Architectural Recommendations / Custom Development.

0.3 Truth Protocol (Mandatory)

This guide obeys a strict truth protocol. Nothing is invented. No regulation, ISO clause, FDA requirement, ANPP requirement, WHO GMP requirement or Odoo feature is fabricated. Where information cannot be verified, the guide writes exactly:

"This information could not be verified from official documentation."

Every statement is labelled with one of four provenance tags:

Field	Explanation
VERIFIED	Confirmed against the codebase (module manifests, model source, security files) or against an official regulation cited inline. Installable and operational as described.
ASSUMPTION	A reasonable inference not contradicting any verified source, but not directly confirmed. Treat as a starting point for validation, not as fact.
ARCHITECTURAL RECOMMENDATION	A design proposal from the Functional Specification or this guide that has no corresponding installed module. Describes intent and target state, not current capability.
CUSTOM DEVELOPMENT	Functionality specified but absent from the codebase. Must be designed, built, validated and maintained before it can be used in a regulated environment.

Every regulatory statement indicates its source. If a statement cannot be mapped to an official regulation it is never presented as mandatory.

0.4 Regulatory Disclaimer

COMPLIANCE IS NOT A SOFTWARE PROPERTY

The Life Sciences Suite is designed to support the implementation of processes aligned with applicable regulatory frameworks. Software alone does not guarantee regulatory compliance. Compliance depends on organisational procedures and policies, system validation (per FDA 21 CFR Part 11, EU GMP Annex 11, or applicable guidance), staff training and competency, the quality management system, and ongoing monitoring and continuous improvement. **No module in this suite is certified as “compliant” with any regulatory framework unless explicitly stated with verified evidence from the relevant regulatory authority.** The regulatory mappings in this guide indicate which processes a module *supports*; they do not imply certification.

Specific verification gaps acknowledged in this guide include: precise ANPP (Algeria) clause numbers, the full Odoo 19 Community module manifest set, and the feature sets of certain OCA repositories. Where such gaps exist, the guide states *“This information could not be verified from official documentation.”* rather than inventing detail.

0.5 Regulatory Frameworks Referenced

Where applicable, implementation steps are mapped to the following official frameworks. Only clauses that can be verified are cited. The list reflects the Functional Specification §3.1.

Table 0.1 — Regulatory frameworks referenced in this guide (Functional Specification §3.1). S = the suite supports implementation of processes aligned with the framework; it does not imply certification.

0.6 How This Guide Is Structured

The guide follows a realistic greenfield pharmaceutical implementation sequence. The full work is delivered in volumes; this Foundation document contains Volume 0 (front matter) and Volume 1 (platform foundation, Steps 1–7). Each implementation step uses a consistent section template:

Table 0.2 — Section template applied to each implementation step (condensed from the 57-section master template).

0.7 Two-Tier Treatment

A literal application of all sections to every step would produce repetitive material of little operational value. This guide therefore uses two tiers:

- **Tier A — Master Configuration Steps** (the steps in the implementation order) receive the full section treatment wherever the section is meaningful. Sections that genuinely do not apply (for example “Generated Documents” for the Users step) are explicitly marked N/A rather than padded.
- **Tier B — Cross-Cutting Reference** topics (FDA 21 CFR Part 11 controls, ALCOA+ data integrity, GS1 serialization, integration architecture, CSV strategy) are covered once, comprehensively, in dedicated reference volumes and cross-referenced from individual steps.

0.8 Edition & Revision Control

Field	Explanation
Document version	1.0 (Draft for review)
Issued	August 2026
Supersedes	None (initial issue)
Next review	Upon completion of validation activities or material change to the configured system
Owner	Life Sciences Suite Architecture Team
Change control	Material changes to this guide are themselves subject to change control (see Volume 2, Step 15) once the QMS is operational.

VOLUME 1

Volume 1 — Foundation & Architecture

Volume 1 establishes the platform on which every subsequent module depends. It begins with an empty database and proceeds through native and OCA module baseline, company/site/location topology, users, the security architecture, and the two cross-cutting controls — electronic signatures and audit trail — that every later GxP record relies upon. No step in Volume 2 or beyond may be started before the corresponding prerequisite in this volume is complete.

1.1 Four-Layer Architecture

The Life Sciences Suite is organised in four layers, each building on the one below. This model is taken from the supplied Architecture document and is confirmed by the module manifests in the codebase.

Table 1.1 — Four-layer architecture, verified against the codebase.

1.2 Deployment Topology

VERIFIED AGAINST CODEBASE

The project ships as a Docker Compose stack: an `odoo:19` application container built from a local Dockerfile, a `postgres:16` database container with a healthcheck, and named volumes for the configuration, addons, Odoo filestore and logs. The application listens internally on port 8069 and is published on host port 8071. Addons under `./addons` are mounted at `/mnt/extra-addons` and declared via `addons_path` in `odoo.conf`.

Field	Explanation
Database	PostgreSQL 16 (container <code>odoo19Claude_1s-db</code>)
Application	Odoo 19 Community (container <code>odoo19Claude_1s</code>)
Host port	8071 → container 8069
Addons path	<code>/mnt/extra-addons</code> (mounted from <code>./addons</code>)
Master password	Stored as a PBKDF2-SHA512 hash in <code>config/odoo.conf</code>
Filestore	<code>./odoo-data</code> (mounted at <code>/var/lib/odoo</code>)
Logs	<code>./logs</code> (mounted at <code>/var/log/odoo</code>)

REGULATORY NOTE

For a GxP system, the deployment topology itself is part of the validated state. The Infrastructure Qualification (IQ) for the platform — containers, database, network, backups — must be documented and version-controlled before any application-level validation begins. This is addressed in Step 1.

STEP 1 · TIER A

Step 1 — Empty-Database Initialization

Module status

VERIFIED

Native Odoo platform (no custom module). Verified against `docker-compose.yml`, `Dockerfile` and `config/odoo.conf`.

1. Business Objective

Bring the application and database containers online from a clean state and create the first (empty) Odoo database, so that all subsequent configuration has a known, reproducible starting point.

2. Business Context

The ERP starts with an empty database. Nothing exists. This step is the single source of truth for the platform: every later step assumes the database created here. Skipping or improvising it (for example by cloning a demo database) makes the validated state unverifiable and must not be done in a regulated environment.

3. Why this step is required

- Establishes a reproducible, traceable platform baseline.
- Records the exact Odoo, PostgreSQL and module versions that will enter validation.
- Produces the first evidence (database manager screen, installed module list) for the Installation Qualification (IQ) of the platform.
- Separates **infrastructure** qualification from **application** configuration, which is a GAMP 5 expectation for category 5 (custom) and category 4 (configured) software.

6. Regulatory Context

REGULATORY NOTE

GAMP 5 (2nd ed.) recommends a risk-based, life-cycle approach to computerised system validation, beginning with a documented and reproducible installed system. EU GMP Annex 11 §4 (Validation) and Annex 15 (Qualification and Validation) require that the system be qualified before use. This step produces the platform-level qualification evidence; module-level IQ/OQ/PQ follow in later steps.

7. Applicable Industries

All industries: Pharmaceutical, Medical Devices, Medical Plastics, Cosmetics, Laboratories.

8. Required Modules

Field	Explanation
Native Odoo	<code>base</code> (installed automatically with the first database). No other module is required at this point.
OCA	None.
Life Sciences	None.

10. Prerequisites

1. Docker Engine and Docker Compose are installed on the host.
2. Host ports 8071 (application) and 5432 (database, if exposed) are free.
3. The project directory `D:\Docker\odoo19Claude_ls-docker` is intact, with `docker-compose.yml`, `Dockerfile`, `config/odoo.conf` and the `addons/` tree.
4. A backup destination for the PostgreSQL volume is identified (see §55).

11. Actors

Field	Explanation
Administrator	Builds and starts the stack; creates the database.
IT / Infrastructure	Owns the host, Docker, network and backups.
Validation	Witnesses and records the platform IQ evidence.

16. Configuration Parameters

The platform is configured through three files. Their effective values, verified from the codebase, are:

Table 1.2 — Platform configuration, verified from the codebase.

PRODUCTION HARDENING (ARCHITECTURAL RECOMMENDATION)

For a production GxP deployment, set `list_db = False` (suppresses the database selector), terminate TLS at a reverse proxy and set `proxy_mode = True`, restrict the database container to the application network only, and enable point-in-time backups for PostgreSQL. These are infrastructure decisions that must be captured in the platform IQ before validation sign-off.

19. Workflow

The end-to-end sequence to bring the platform online and create the empty database:

1. **Build & start:** from the project root, run `docker compose up -d --build`. The Postgres container becomes healthy; the Odoo container starts and waits for it.
2. **Reach the database manager:** open `http://:8071/web/database/manager`.
3. **Create the database:** choose *Create Database*, enter the database name, the master password (matching the hash in `odoo.conf`), the administrator password, language and country. **Do not** load demo data for a validated system.

4. **Confirm the baseline:** log in as admin; the installed module list contains only `base`. This is the empty-database baseline.
5. **Record evidence:** capture the database manager screen, the installed module list, the Odoo and PostgreSQL versions, and the image digests. These form the first IQ artefacts.

39. Common Mistakes

- **Loading demo data** in a database intended for validation. Demo data pollutes master data and audit trails. Always create without demo data.
- **Cloning a dev database** into the validation environment. The provenance of every record must be traceable; clones break this.
- **Exposing `list_db = True`** in production — leaks the database list and the create/restore/delete controls.
- **Reusing the default admin password** from `odoo.conf`. Rotate it immediately after database creation and record the rotation.

40. Best Practices

- Tag the exact image digests in the IQ pack so the platform can be rebuilt byte-for-byte.
- Keep `odoo.conf`, `docker-compose.yml` and the Dockerfile under version control; treat changes as platform change-control events.
- Run the database and application clocks from the same NTP source. Timestamp integrity underpins every audit trail in the suite.

47–51. Testing & Validation (Platform IQ)

The platform Installation Qualification establishes that the system is installed correctly and is reproducible. The minimum evidence set:

Table 1.3 — Platform IQ evidence set.

55. Backup Strategy

Before any further configuration, establish the platform backup: a PostgreSQL logical backup (`pg_dump`) of the fresh empty database plus a copy of the filestore and `addons/` tree. This proves the restore path works while the system is trivially small. See Volume 6 for the full backup & disaster-recovery strategy.

57. References

- Supplied Architecture document, §1–4.
- Supplied Functional Specification §4 (Architecture Overview), §5 (Native modules).
- `docker-compose.yml`, Dockerfile, `config/odoo.conf` (codebase).
- GAMP 5, 2nd edition (ISPE) — life-cycle & risk-based validation.
- EU GMP Annex 11 §4 (Validation); Annex 15 (Qualification and Validation).

STEP 2 · TIER A

Step 2 — Native Module Baseline Install

Module status

VERIFIED

Native Odoo 19 Community modules. Dependency order per Functional Specification §15.1.

1. Business Objective

Install the Layer 1 native modules in dependency order, so that the ERP foundation (sales, purchase, inventory, manufacturing, accounting, HR, projects, maintenance) is present and the OCA and Life Sciences layers can be installed on top.

3. Why this step is required

Every OCA and Life Sciences module depends on one or more native modules (see the dependency matrix, Functional Specification §15). Installing native modules out of order produces broken or partially-installed dependencies that surface later as obscure errors during validation.

8. Required Modules & Dependency Order

Install in this order, derived from Functional Specification §15.1:

Table 1.4 — Native module install order (Functional Specification §15.1).

VERIFIED AGAINST CODEBASE

The native `quality` module folder is **not** present in the project `addons/` tree; it is part of the Odoo 19 Community core image and is installed from there. The OCA module `quality_control_oca` is present in `addons/` and is addressed in the OCA selection chapter (§1.3).

14. Menus & 15. Navigation

After installation, the main menu gains the installed applications. From **Apps** → search the technical name → click *Install*. Confirm each install completes without warnings in **Settings** → **Technical** → **Modules**.

18. Business Rules

- Install `account` before `sale` and `purchase` — both depend on it.
- Install `hr` before `maintenance` — maintenance equipment references employees and departments.
- Do not install `mrp` before `stock`.

39. Common Mistakes

- Installing all native modules at once via the Apps menu without checking dependencies — leads to partial installs.

- Installing Enterprise-only modules (documents, approvals, AI features) which are **not** available in Community Edition. The Functional Specification §5.1 notes these are Enterprise-only.

47–51. Testing & Validation

For each native module, capture: install timestamp, version, and a smoke test (open the module's main menu, create one record of the primary model, delete it). This is the module-level IQ foundation that the OCA and Life Sciences layers inherit.

57. References

- Functional Specification §5 (Native modules), §15.1 (Layer 1 dependency matrix).
- Odoo 19 Community documentation — module manifests and dependencies.

CHAPTER · TIER B REFERENCE

1.3 OCA Module Selection & Deployment

Layer 2 selects OCA modules that strengthen the platform where native Odoo Community is thin: management-system documentation, controlled pages, quality control points, audit logging and maintenance extensions. This chapter inventories what is **verified present** in the codebase and recommends a baseline set. OCA modules are AGPL-3 unless otherwise noted.

Verified OCA families present

VERIFIED AGAINST CODEBASE

The following OCA families are present under `addons/` and are directly relevant to the Life Sciences Suite:

Table 1.5 — OCA modules verified present and relevant to the suite.

Recommended baseline

For a pharmaceutical greenfield, install the following OCA baseline before the Life Sciences Core layer. Each is justified and tagged:

Field	Explanation
mgmtsystem + mgmtsystem_nonconformity + mgmtsystem_action	Provides the nonconformity / action backbone. The <code>ls_capa</code> module can integrate with or supersede this depending on configuration choice. Decision required: adopt OCA <code>mgmtsystem</code> as the QMS backbone, or the custom <code>ls_qms/ls_capa</code> . See Volume 2.
document_page + document_page_approval	Controlled document pages with approval workflow. Useful for the Quality Manual and early SOPs while <code>ls_document_management</code> is being validated.
quality_control_oca	In-process quality control points. Pairs with <code>ls_pharma</code> batch records (Step 26).
base_maintenance + maintenance_plan + maintenance_e quipment_certification	Equipment and preventive-maintenance foundation for calibration (Step 17) and equipment qualification (Step 24).
auditlog	Generic audit log; retained as a defence-in-depth alongside <code>ls_audit_trail</code> (Step 7).

VERIFICATION GAP — ASSUMPTION

The Functional Specification §6.7 states that the OCA module lists for `account-analytic`, `crm` and `agreement` could not be fully verified from official documentation. Before relying on any OCA module in a validated process, confirm its manifest, license and Odoo 19 compatibility directly in the OCA repository.

57. References

- Functional Specification §6 (OCA modules), §15.2 (Layer 2 dependency matrix).
- OCA repositories: <https://github.com/OCA>.

STEP 3 · TIER A

Step 3 — Company, Sites, Plants, Buildings, Cleanrooms, Warehouses, Locations

Module status

VERIFIED

Native **base** (res.company) and **stock** (stock.warehouse, stock.location). Cleanroom grades are represented via location structure and naming.

1. Business Objective

Model the physical and organisational topology of the company: legal entities, sites, plants, buildings, cleanrooms, warehouses and storage locations. This topology is the substrate for inventory, manufacturing, quality, environmental monitoring and traceability.

6. Regulatory Context

REGULATORY NOTE

EU GMP Annex 1 (manufacture of sterile medicinal products) classifies cleanroom environments by grade (A/B/C/D). EU GMP Part I §3 (Premises) requires premises to be located, designed, constructed and maintained to suit the operations. The location hierarchy in Odoo is the system's representation of these premises and must correspond to the site master plan and environmental classification drawings.

16. Configuration Parameters & 17. Field-by-field

The topology is built from three native objects. Recommended mapping:

Table 1.6 — Topology mapping.

Field	Explanation
res.company — name	Legal entity name (e.g. the manufacturing company).
res.company — currency	Base currency; multi-currency configured in account.
res.company — parent_id	Parent company, for group structure.
stock.warehouse — name / code	Site/plant name and short code (e.g. PLT01).
stock.warehouse — company_id	Owning legal entity.
stock.location — name	Location full name; encode building/grade/bin.
stock.location — location_id	Parent location → builds the hierarchy.
stock.location — usage	internal / view / supplier / customer / transient / inventory / production.
stock.location — company_id	Owning company (for multi-company isolation).
stock.location — active	Archive (not delete) decommissioned locations to preserve history.

18. Business Rules

- Cleanroom grades are represented by location, not by a dedicated model. Enforce a naming convention so environmental monitoring (`ls_environmental_monitoring`, Step 19) can reference locations unambiguously.
- Quarantine locations must be segregated from released-goods locations to support batch hold/release (Step 28).
- Decommissioned premises are archived, never deleted — deletion destroys historical traceability.

40. Best Practices

- Mirror the site master plan and classification drawings exactly; the location tree is the system's authoritative site model.
- Use view-type locations to group buildings/floors; keep stock quants only on leaf internal locations.
- Record the mapping (location → physical room → GMP grade) in a controlled document; it is inspection evidence.

57. References

- Functional Specification §5.2.4 (Inventory / stock), §5.2.1 (base).
- EU GMP Annex 1 (sterile manufacture) and Part I §3 (Premises).

STEP 4 · TIER A

Step 4 — Departments & Users

Module status

VERIFIED

Native hr (hr.department, hr.employee, hr.job) and base (res.users, res.groups).

1. Business Objective

Create the organisational structure (departments, jobs, employees) and the user accounts that will operate the system, each linked to an employee. This step provides the actors referenced by every later workflow.

16–17. Fields

Field	Explanation
hr.department — name / parent_id	Department and its parent (org tree).
hr.department — manager_id	Department head (employee).
hr.employee — name / user_id	Employee and the linked login user (one user per employee in a regulated system).
hr.employee — department_id / job_id	Assignment to department and role.
hr.employee — work_email / work_phone	Contact details used by notifications.
res.users — login	Unique login; recommend employee-style naming.
res.users — email	Used for password reset and notifications.
res.users — groups_id	Security groups; assigned in Step 5.
res.users — company_ids / company_id	Allowed and default companies.
res.users — share	False for internal users; True for portal/external.

18. Business Rules

- One user per employee. Shared logins destroy attribution and break ALCOA+ (see Volume 6).
- Deactivate, never delete, leavers — deletion orphans audit trails and signatures.
- Internal users only for GxP operations; portal/external users for non-GxP access.

39. Common Mistakes

- Reusing a generic qa / admin login across a team.
- Granting Administrator rights broadly. Reserve Settings access to a small, named administrator group.

57. References

- Functional Specification §5.2.8 (Employees / hr), §5.2.1 (base).
- ALCOA+ data-integrity principles — see Volume 6.

STEP 5 · TIER A

Step 5 — Security Architecture: Groups, Roles, Access Rights, Record Rules

Module status

VERIFIED

Native base (`res.groups`, `ir.model.access`, `ir.rule`). Odoo 19 uses the new `res.groups.privilege` model (verified in `ls_signature_groups.xml`).

1. Business Objective

Define a least-privilege security architecture: who can read, write, create and delete each object, scoped by company and by record rule. This is the control foundation for segregation of duties and for ALCOA+ attribution.

6. Regulatory Context

REGULATORY NOTE

FDA 21 CFR Part 11 §11.10(d) and §11.300 require controls over access, including authority checks ensuring that only authorised individuals can use the system, electronically sign, or access records. EU GMP Annex 11 §7 (Data Security) requires access control. Segregation of duties is a general GMP expectation (e.g. batch release independent of production).

17. Field-by-field — the four security mechanisms

Table 1.7 — The four security mechanisms.

VERIFIED AGAINST CODEBASE

Odoo 19 changed `res.groups.category_id` to `res.groups.privilege_id`, pointing at the new `res.groups.privilege` model. This is confirmed in the codebase: `ls_electronic_signature/security/ls_signature_groups.xml` declares a `module_category_life_sciences` and a `privilege_ls_signature`, then groups reference the privilege. **The audit_trail module deliberately does not assign a category**, with a comment that the field name “could not be verified for Odoo 19 from official documentation” — it relies on group name and `implied_ids` only. This is an ASSUMPTION-level gap to watch during upgrade.

32. Security Groups & 33. Record Rules — design pattern

Each Life Sciences module follows the same pattern, verified in the code:

- A module category under **Life Sciences** groups the module's roles.
- Role groups are ordered by privilege and chained with `implied_ids` so each higher role implies the lower (e.g. Manager → Auditor → Viewer → Signer).
- Record rules scope records to the user's company (multi-company isolation).

- Audit/signature evidence groups are **read-only by design** so auditors cannot alter the evidence they audit (segregation of duties).

40. Best Practices

- Document the role-to-user mapping in the QMS; it is inspection evidence.
- Run periodic access reviews (a native or scheduled action) and record the review.
- Reserve Administrator for infrastructure work; day-to-day QA/QC/Production work uses named functional roles.

57. References

- FDA 21 CFR Part 11 §11.10, §11.300; EU GMP Annex 11 §7.
- Functional Specification §7.x (per-module Security Groups).
- `ls_electronic_signature/security/ls_signature_groups.xml`,
`ls_audit_trail/security/ls_audit_trail_groups.xml`.

STEP 6 · TIER A

Step 6 — Electronic Signatures

Module status

VERIFIED

ls_electronic_signature v19.0.1.0.0, AGPL-3, status Beta. Manifest and models verified in the codebase. Depends on base, mail.

1. Business Objective

Provide electronic signature capability with signature manifestations, signature/record linking, and identification-code-and-password controls, to support FDA 21 CFR Part 11 electronic signature requirements.

6. Regulatory Context

REGULATORY NOTE

FDA 21 CFR Part 11 is the primary driver: §11.50 (signature manifestations — the signed record must show the printed name, the date and time, and the *meaning* of the signature); §11.70 (signature / record linking — signatures must be inextricably linked to their record); §11.200 (two distinct identification-code components for non-biometric signatures); §11.300 (controls for identification codes/passwords, including detection of unauthorised use). EU GMP Annex 11 §2.4-2.5 expects equivalent controls for electronic signatures.

8. Required Modules

Field	Explanation
Native Odoo	base, mail
OCA	None.
Life Sciences	ls_electronic_signature (VERIFIED present, Beta).

12. Roles and Permissions (verified)

VERIFIED AGAINST CODEBASE

The module declares five roles under the **Life Sciences** category and the **Electronic Signatures** privilege, chained by `implied_ids`. The source comments cite 21 CFR 11.300(d) for the Security Unit role.

Table 1.8 — Electronic signature roles (verified in source).

13. Required Master Data

- **Signature Meanings** — the controlled vocabulary of meanings (e.g. AUTHORED, REVIEWED, APPROVED, REJECTED), per 21 CFR 11.50(a)(3).
- **Signature Policies** — rules binding meanings to models/groups.
- **Users** with the Signer role, each linked to an employee (Step 4).

14–15. Menus & Navigation

Verified menu structure (from the manifest data files):

- **Security** → **Electronic Signatures** — requests, logs, sessions, attempts.
- **Security** → **Signature Logs** — the immutable signature history.
- **Security** → **Integrity Checks** — verification of signature evidence.
- Configuration (meanings, policies) under Settings / the module's config views.

16–17. Configuration & Field-by-field — Signature Meaning

Signature Meaning is the controlled-vocabulary object required by 21 CFR 11.50(a)(3). The model is `ls.signature.meaning`; the following fields are verified in the source (`ls_signature_meaning.py`):

Field	Explanation
name	Meaning wording shown on screen and on printed signature manifestations (e.g. 'Approved by'). Translatable.
code	Stable technical identifier for reports and integrations. Validated by regex <code>^[A-Z][A-Z0-9_]{1,31}\$</code> (2–32 chars, upper-case letter, digits, underscores). Unique per company.
sequence	Order in selection lists.
description	When this meaning applies; shown to the signer in the wizard.
require_reason (Reason Mandatory)	If enabled, the signer must record a free-text reason. Used for negative decisions such as rejection.
company_id	Owning company; the meaning is company-specific.
group_ids (Authorised Groups)	Groups whose members may use this meaning. Empty = any user with the Signer role.
active	Archive (not delete) to preserve history.

18. Business Rules (verified)

- A meaning code must be unique per company (DB constraint `UNIQUE(code, company_id)`).
- The code format is enforced at validation; invalid codes raise a `ValidationError`.
- A meaning is available to a user only if it has no authorised groups, or the user is a member of one of them (`is_available_to()`).

22–23. Electronic Signatures & Audit Trail

Each executed signature is written to the immutable signature log with the printed name, the date/time, the meaning and (if required) the reason, satisfying 21 CFR 11.50. Signature/record linking (21 CFR 11.70) is provided by the signature mixin attached to signed models. Attempts — including failed/unauthorised attempts — are recorded and notify the Security Unit group (21 CFR 11.300(d)). The full audit trail of signature configuration changes is handled by `ls_audit_trail` (Step 7).

24. Notifications

- Security Unit is notified immediately on attempted unauthorised use of an identification code or password (21 CFR 11.300(d)).
- Signature request workflows notify the requested signer(s).

25–27. Documents, Records, Reports

- **Signature manifestation** report — renders name, date/time, meaning (21 CFR 11.50).
- **Signature certificate** report — per-signature evidence.
- **Integrity check** records — periodic verification of the signature evidence store.

47–52. Testing & CSV

Minimum validation scope for the electronic signature module (21 CFR Part 11 / Annex 11):

Table 1.9 — Electronic signature validation scope.

39. Common Mistakes

- Granting the Manager role write/delete on signature records — the module deliberately withholds this; do not override it.
- Letting users share identification codes/passwords. Each signer is an individual.
- Treating the signature log as editable. It is immutable evidence.

57. References

- FDA 21 CFR Part 11 §11.50, §11.70, §11.200, §11.300.
- EU GMP Annex 11 §2.4–2.5.
- `ls_electronic_signature` — `manifest`, `models/ls_signature_meaning.py`, `security/ls_signature_groups.xml`.
- Functional Specification §7.14 (Electronic Signatures).

STEP 7 · TIER A

Step 7 — Audit Trail

Module status

VERIFIED

`ls_audit_trail` v19.0.1.0.0, AGPL-3. Depends on `base`, `mail`. Verified: `manifest`, `models` (`ls_audit_trail_rule.py`, `ls_audit_trail_log.py`, `evidence pack`, `verification`), `security groups`, `record rules`, `crons`.

1. Business Objective

Provide a tamper-evident, field-level audit trail for GxP-regulated Odoo data, capturing who changed what, when, from what to what, and why — with integrity verification and exportable evidence packs.

6. Regulatory Context

REGULATORY NOTE

FDA 21 CFR Part 11 §11.10(e) requires secure, computer-generated, time-stamped audit trails recording the date and time of operator entries and actions that create, modify, or delete electronic records, without obscuring previous values. **EU GMP Annex 11 §9 (Audit Trails)** requires audit trails generated by the system, with change and deletion reasons, retained for the record retention period and available for review. ALCOA+ (Attributable, Legible, Contemporaneous, Original, Accurate + Complete, Consistent, Enduring, Available) underpins both.

8. Required Modules

Field	Explanation
Native Odoo	<code>base</code> , <code>mail</code>
OCA	<code>auditlog</code> (VERIFIED present) — generic field-level logging; retained as defence-in-depth. The regulated trail of record is <code>ls_audit_trail</code> .
Life Sciences	<code>ls_audit_trail</code> (VERIFIED).

12. Roles and Permissions (verified)

VERIFIED AGAINST CODEBASE

Three roles, chained by `implied_ids`. The module deliberately omits a category (ASSUMPTION: Odoo 19 field-name uncertainty) and relies on group name + implied ids.

Table 1.10 — Audit trail roles (verified in source).

13. Required Master Data

- Audit rules** — for each regulated model, which fields and which operations (create/write/unlink) are audited, and the justification.
- Company scoping per rule.

16–17. Configuration & Field-by-field — Audit Rule

The audit rule (`ls.audit_trail.rule`) is the configuration object of the engine. Verified fields from `ls_audit_trail_rule.py`:

Field	Explanation
<code>name</code>	Free-text label shown in the rule list. Tracked in the rule's own chatter.
<code>active</code>	Archived rules are ignored by the engine.
<code>model_id</code> (Audited Model)	The <code>ir.model</code> whose records are audited. Cascade-deleted with the model.
<code>model_name</code>	Related stored technical name; indexed for fast lookups.
<code>company_id</code>	Empty = applies to every company; set = applies only to that company's users.
<code>log_create</code> / <code>log_write</code> / <code>log_unlink</code>	Audit creations / modifications / deletions. At least one must be on (DB-enforced).
<code>field_ids</code> (Audited Fields)	Fields written to the trail. Empty = audit every stored field except the technical fields <code>id/create_uid/create_date/write_uid/write_date/display_name</code> .
<code>excluded_field_ids</code> (Excluded Fields)	Fields never audited, even when Audited Fields is empty. Use for high-volume or confidential fields.
<code>entry_count</code>	Computed count of audit entries for the audited model.
<code>note</code> (Justification)	Documented reason for the scope. Regulated organisations are expected to justify why a model/field is audited or excluded.

18. Business Rules (verified)

- **One rule per model per company** — DB constraint `UNIQUE(model_id, company_id)`.
- **At least one operation** must be audited — enforced at validation.
- **Fields must belong to the model** — foreign fields are rejected.
- **The engine never audits itself**: models in `NON_AUDITABLE_MODELS` (the audit engine's own models) are rejected, so a configuration change cannot recurse into the engine being reconfigured.
- **Abstract and transient models are rejected** — abstract models own no table; transient records are cleared by the vacuum job and are not part of the regulated record set.
- **Rule changes clear the cached audit configuration** so new rules take effect immediately.
- **Rule changes are recorded in the rule's own chatter, not in the audit log** — by design, to avoid self-recursion.

VERIFIED DESIGN DECISION — SEGREGATION OF DUTIES

The Administrator role can configure rules and run retention; the Auditor role can read, verify and export evidence but cannot configure. The source comment explicitly requires these be held by different people. This is a textbook GxP segregation-of-duties control and should be reflected in the role-to-user mapping recorded in the QMS.

24. Notifications & 26. Generated Records

- Audit entries: `ls.audit_trail.log` (header) + `ls.audit_trail.log.line` (field-level old/new values).
- Integrity verifications: `ls.audit_trail.verification`.

- Evidence packs: `ls.audit_trail.evidence_pack` — tamper-evident, exportable bundles for inspectors.
- Retention runs: scheduled, approved purges past the retention period (Annex 11 §9 retention).

27. Reports

- Audit log report (filterable by model, user, date, operation).
- Evidence pack report — the inspection-ready extract.
- Integrity verification report.

47–52. Testing & CSV

Table 1.11 — Audit trail validation scope.

39. Common Mistakes

- Auditing transient/wizard models — they are cleared by the vacuum job and are not part of the regulated record set (the engine rejects them).
- Auditing every field of a high-volume model without an exclusion list — bloats the trail and slows verification.
- Allowing the same person to hold both Auditor and Administrator roles — breaks segregation of duties.
- Editing audit log records directly in the database — destroys integrity and is detectable by the verification job.

40. Best Practices

- Justify every rule's scope in the `note` field; inspectors ask *why* a field is or is not audited.
- Run the integrity verification cron regularly and review its results.
- Test the restore-and-replay of an evidence pack during periodic review.

55–56. Backup & Disaster Recovery

The audit trail is part of the regulated record set and must be included in the validated backup. Treat evidence packs as additional, tamper-evident snapshots suitable for off-site/immutable storage. See Volume 6 for the full strategy.

57. References

- FDA 21 CFR Part 11 §11.10(e); EU GMP Annex 11 §9 (Audit Trails).
- ALCOA+ data-integrity principles (Volume 6).
- `ls_audit_trail` — manifest, `models/ls_audit_trail_rule.py`, `models/ls_audit_trail_log.py`, `security/ls_audit_trail_groups.xml`.
- Functional Specification §7.15 (Audit Trail).